

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА Российский технологический университет»

РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКИМ РАБОТАМ

по дисциплине «Моделирование сред и разработка приложений виртуальной и дополненной реальности»

Итоговый отчет по практическим работам

Студент группы *ИКБО-50-23, Астахов С.П.*

_____
(подпись)

Старший преподаватель *Горбатов Г.В.*

_____
(подпись)

Отчет представлен «___»_____202__г.

Москва 2026 г.

Содержание

Практическая работа №1 3

Практическая работа №2 7

Практическая работа №3 12

Практическая работа №4 14

Практическая работа №5 16

Практическая работа №1

Создадим сцену, игрока, другие объекты окружения и настроим их, а также сделаем движения главному герою.

Листинг 1 - Код скрипта moveTransform.cs на C#

```
using UnityEngine;

public class moveTransform : MonoBehaviour
{
    private float speed = 20f;
    private float jumpSpeed = 150f;
    //private float yaw = 0.0f;
    //private float pitch = 0.0f;
    void Start()
    {

    }

    void Update()
    {
        if (Input.GetKey(KeyCode.Space) && transform.position.y <= 1f) {
            transform.position += new Vector3(0, jumpSpeed * Time.deltaTime,
0);
        }
        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");
        Vector3 movement = new Vector3(horizontal, 0, vertical);
        transform.position += movement * speed * Time.deltaTime;


    }
}
```

Листинг 2 - Код скрипта Gun на C#

```
using UnityEngine;

public class moveTransform : MonoBehaviour
{
    private float speed = 20f;
    private float jumpSpeed = 150f;
    //private float yaw = 0.0f;
    //private float pitch = 0.0f;
    void Start()
    {

    }

    void Update()
    {
        if (Input.GetKey(KeyCode.Space) && transform.position.y <= 1f) {
            transform.position += new Vector3(0, jumpSpeed * Time.deltaTime,
0);
        }
        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");
        Vector3 movement = new Vector3(horizontal, 0, vertical);
        transform.position += movement * speed * Time.deltaTime;


    }
}
```

Листинг 3 - Код скрипта Bullet на C#

```
using System.Collections;
using UnityEngine;

[RequireComponent(typeof(Rigidbody))] // Запросить компонент Ragidbody
public class Bullet : MonoBehaviour
{
    [SerializeField] private float speed;
    [SerializeField] private float destroyDistance;
    [SerializeField] private Rigidbody rigidbody;

    public void Init(Vector3 startPosition, Vector3 direction)
    {
        if (rigidbody == null)
            rigidbody = GetComponent<Rigidbody>();
        transform.position = startPosition;
        rigidbody.linearVelocity = direction * speed;


        float lifiTime = destroyDistance / speed;

        StartCoroutine(AutoDestroy(lifiTime));


    }
    private IEnumerator AutoDestroy(float lifetime)
    {
        yield return new WaitForSeconds(lifetime);
        Destroy(gameObject);
    }
}
```

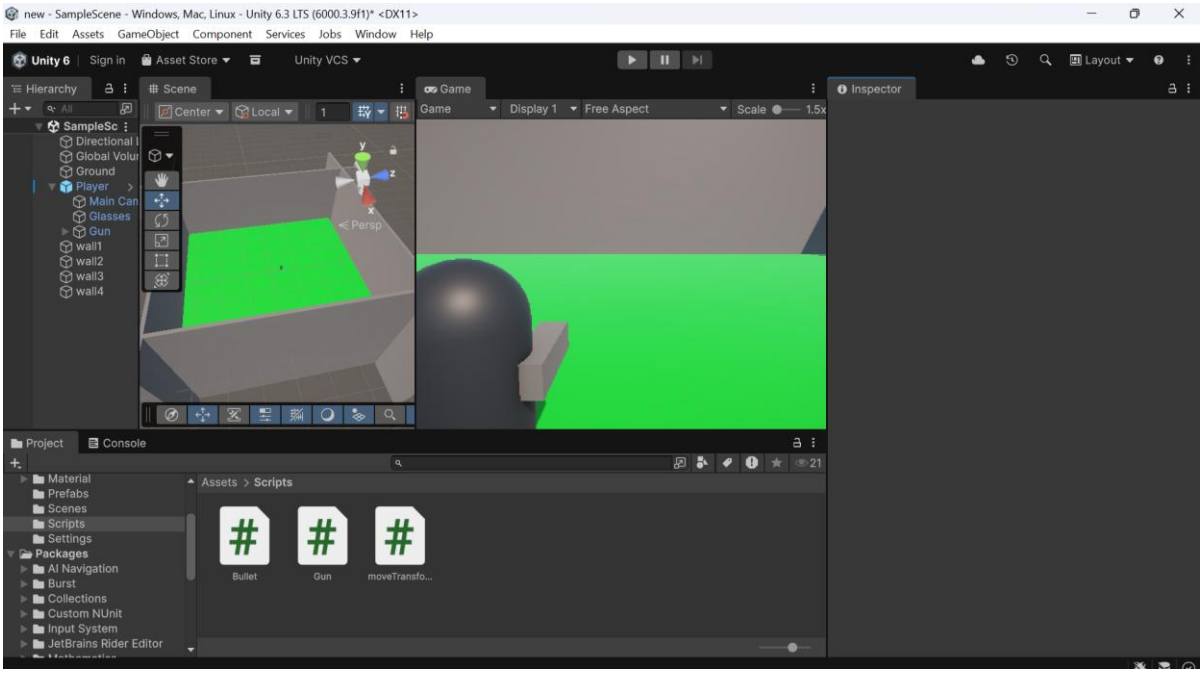


Рисунок 1 — Скриншот сцены Unity

Практическая работа №2

В данной практической работе мы создадим на сцене противника красного цвета и настроим его, чтоб он атаковал нас при приближении.

Листинг 4 - Код скрипта enemy на C#

```
using UnityEngine;

public class Enemy : MonoBehaviour
{
    [Header("Enemy Settings")]
    public float moveSpeed = 3f;
    public float detectionRange = 10f;
    public Transform player;

    private Rigidbody rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        if (rb == null)
            rb = gameObject.AddComponent<Rigidbody>();

        rb.freezeRotation = true;

        // Если игрок не привязан, ищем по тегу
        if (player == null)
            player = GameObject.FindGameObjectWithTag("Player").transform;
    }

    void FixedUpdate()
    {
        // Проверяем расстояние до игрока
        float distance = Vector3.Distance(transform.position, player.position);

        if (distance <= detectionRange)
        {
            // Движение к игроку
            Vector3 direction = (player.position - transform.position).normalized;
            direction.y = 0; // Не двигаемся по вертикали
```

```
            Vector3 targetVelocity = direction * moveSpeed;
            targetVelocity.y = rb.linearVelocity.y;
            rb.linearVelocity = targetVelocity;

            // Поворачиваем врага в сторону игрока
            if (direction != Vector3.zero)
            {
                Quaternion targetRotation = Quaternion.LookRotation(direction);
                transform.rotation = Quaternion.Slerp(transform.rotation, targetRotation, Time.fixedDeltaTime * 5f);
            }
        }
        else
        {
            // Если игрок далеко - стоим на месте
            rb.linearVelocity = new Vector3(0, rb.linearVelocity.y, 0);
        }
    }

    // Визуализация радиуса обнаружения
    private void OnDrawGizmosSelected()
    {
        Gizmos.color = Color.red;
        Gizmos.DrawWireSphere(transform.position, detectionRange);
    }
}
```

Листинг 5 - Код скрипта Mouse на C#

```
using UnityEngine;

public class Mouse : MonoBehaviour
{
   [SerializeField]
   protected float mouseSensitiviti = 1.5f;
   [SerializeField]
   protected float smooth = 10;
   [SerializeField]
   public Transform characher;
   float xRotation;
   float yRotation;
   void Start()
   {
      Cursor.lockState = CursorLockMode.Locked;
   }

   void Update()
   {
      yRotation += Input.GetAxis("Mouse X") * mouseSensitiviti;
      xRotation -= Input.GetAxis("Mouse Y") * mouseSensitiviti;

      xRotation = Mathf.Clamp(xRotation, -90f, 90f); // Ограничение просмотра выше 90гр

   }

   protected void RotateCharacter()
   {
      transform.rotation = Quaternion.Lerp(transform.rotation, Quaternion.Euler(0, yRotation, 0), Time.deltaTime *
smooth);
      characher.rotation = Quaternion.Lerp(characher.rotation, Quaternion.Euler(0, yRotation, 0), Time.deltaTime *
smooth);
   }
}
```

Листинг 6 - Код скрипта FollowObjectView на C#

```
using UnityEngine;

public class FollowObjectView : MonoBehaviour
{
    [SerializeField]
    private Transform fpsTransformView;

    void Update()
    {
        transform.position = fpsTransformView.position;
        transform.rotation = fpsTransformView.rotation;
    }
}
```

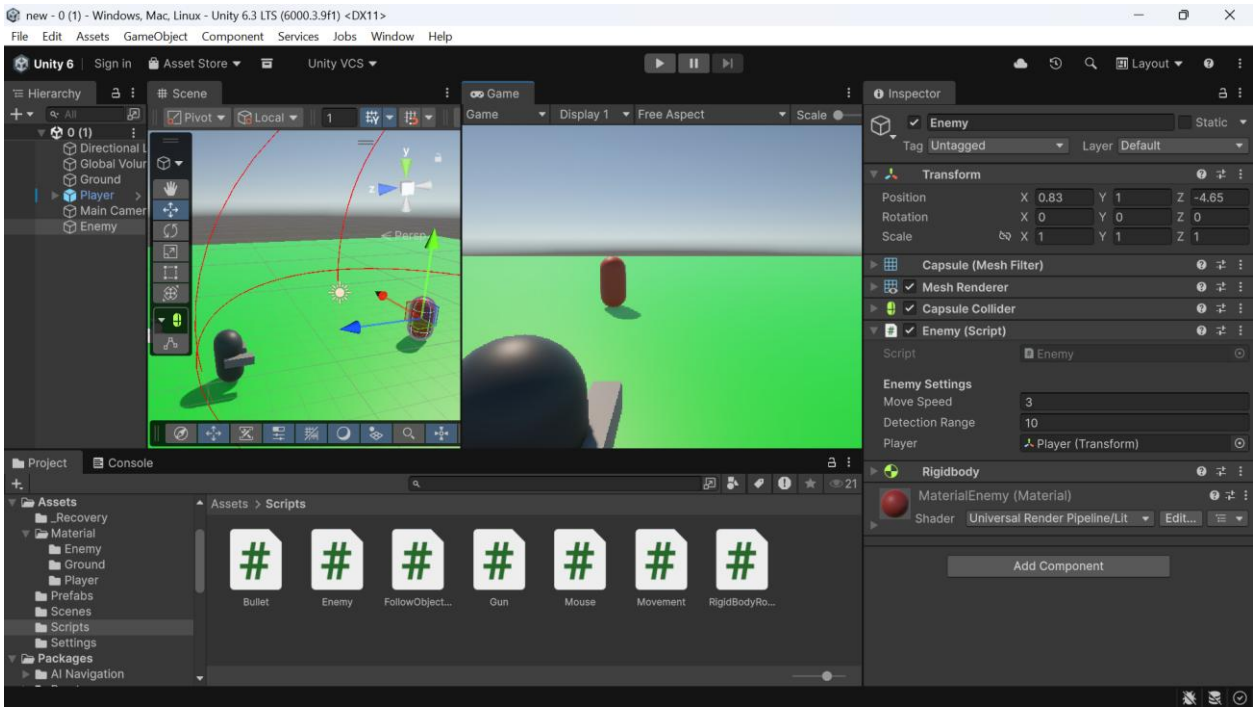


Рисунок 2 — Скриншот Unity противник

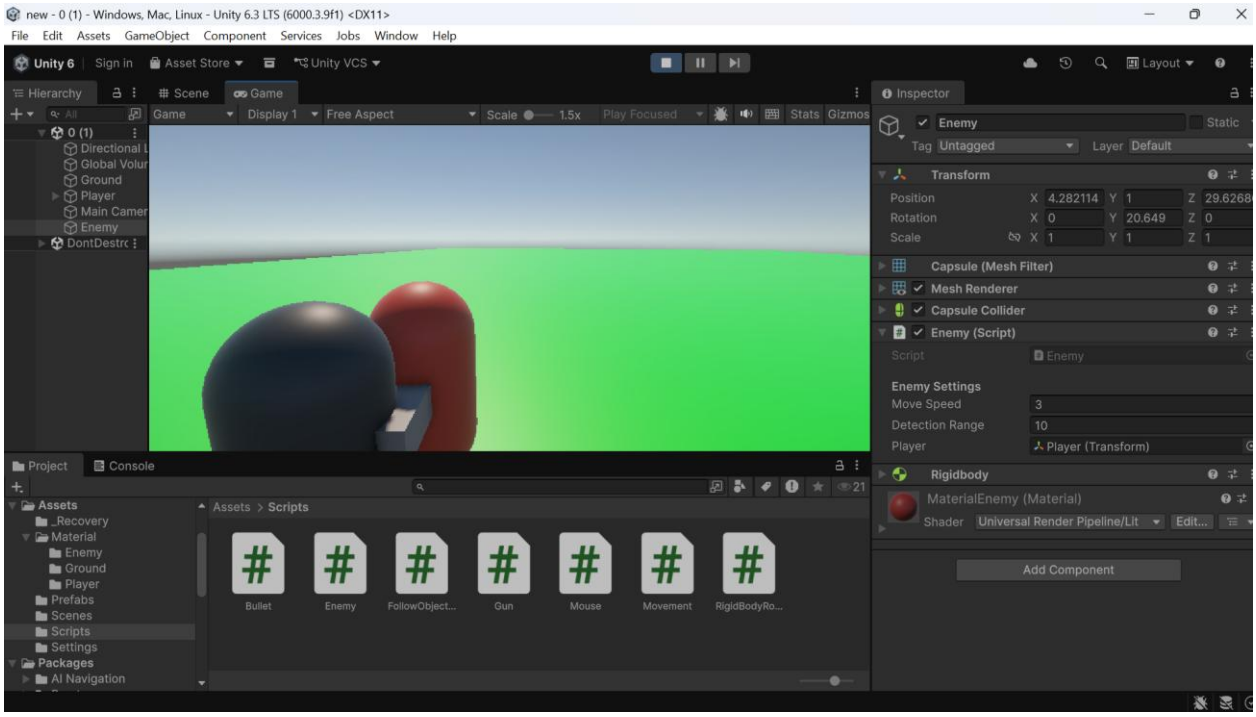


Рисунок 3 — Скриншот Unity противник

Практическая работа №3

В данной практической работе мы создадим переход на разные сцены, а также главное меню и меню с паузой.

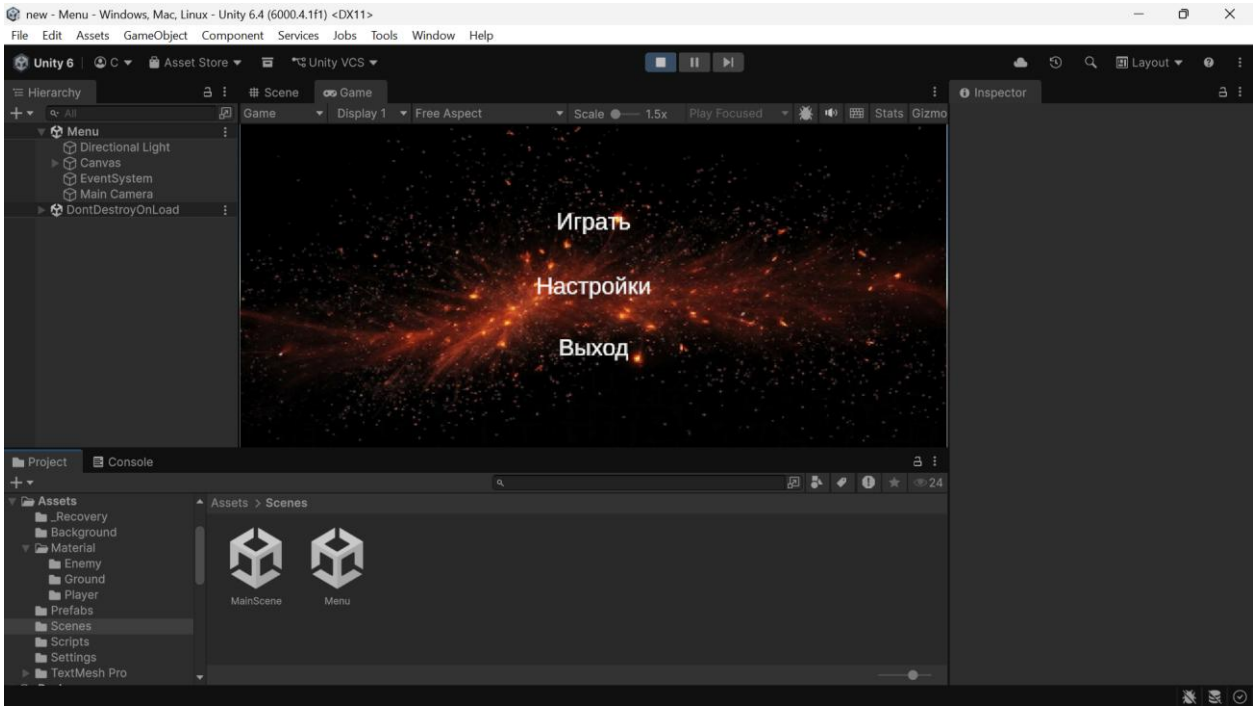


Рисунок 4 — Главное меню игры

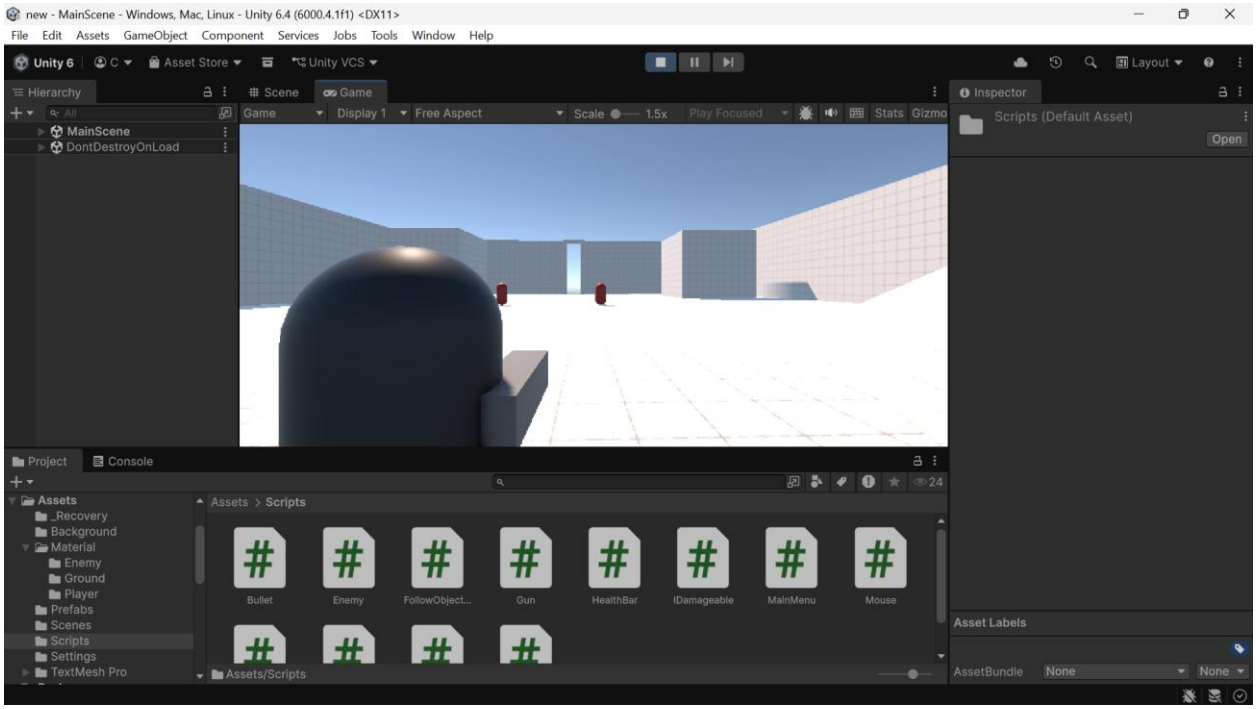


Рисунок 5 — Изменение локации под уровень

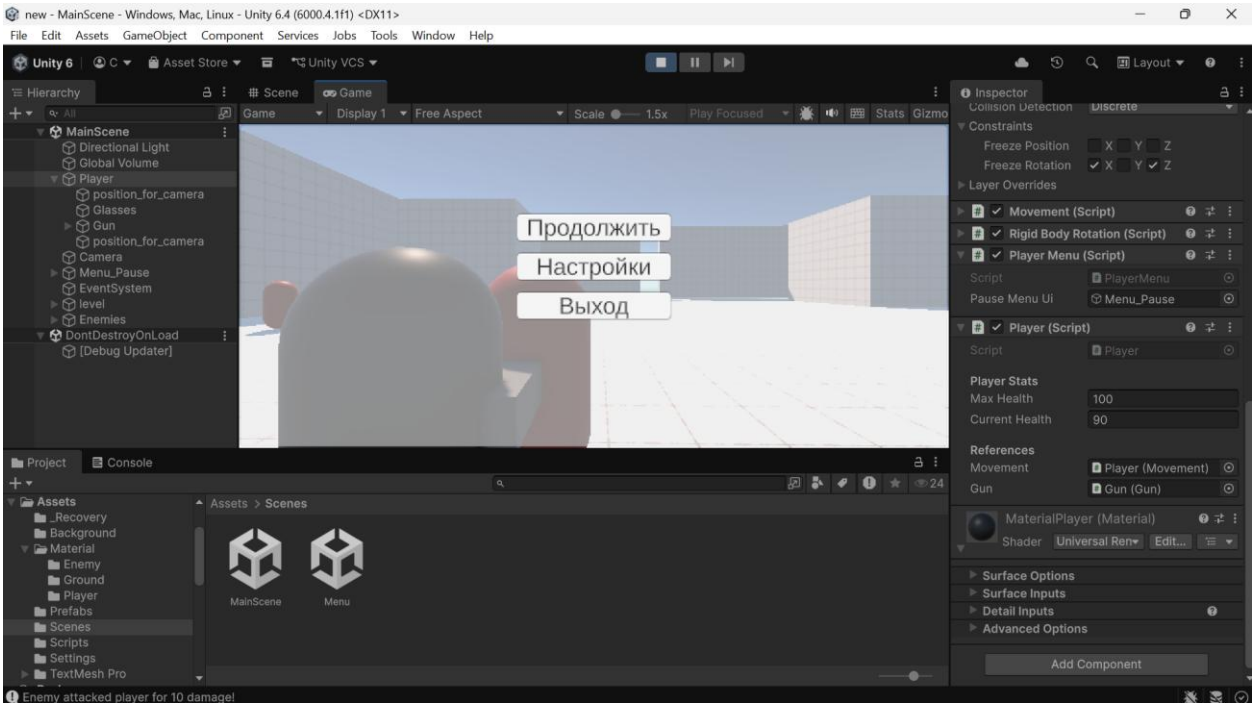


Рисунок 6 — Менюшка паузы

Практическая работа №4

В данной практической работе мы сможем добавить анимации, а также рассмотреть состояния, когда их применять и менять.

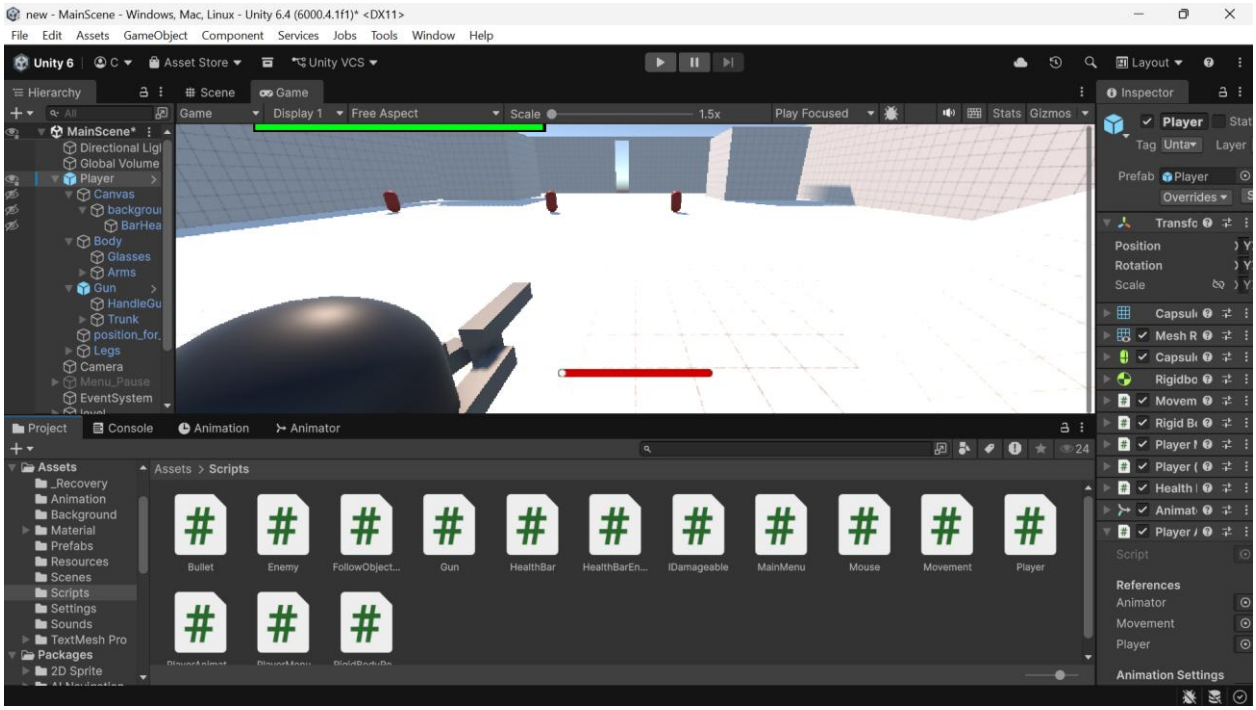


Рисунок 7 —Правка держание оружия

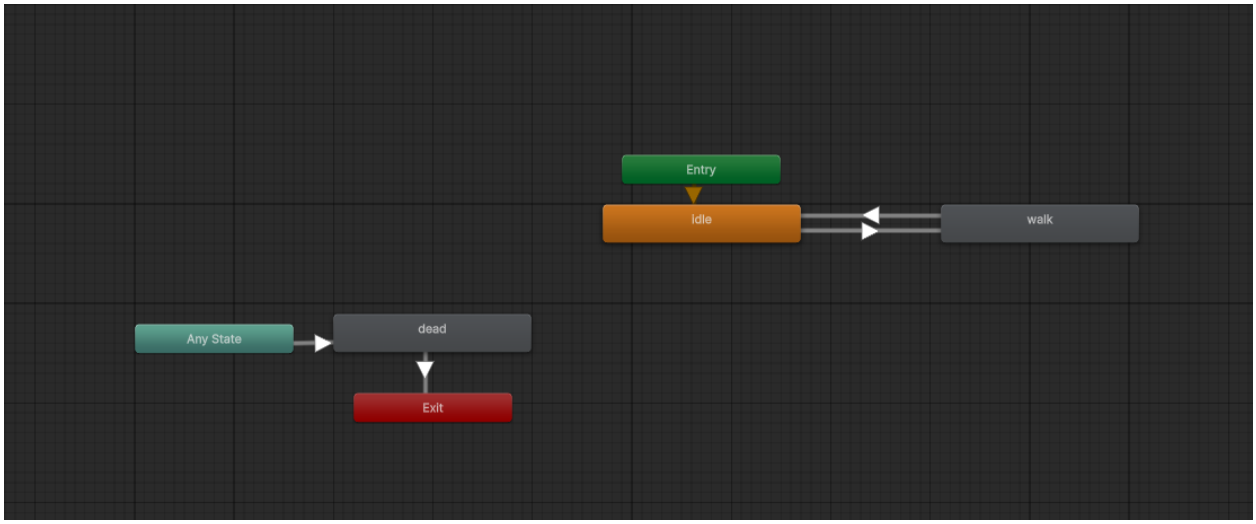


Рисунок 8 — Animator на Unity

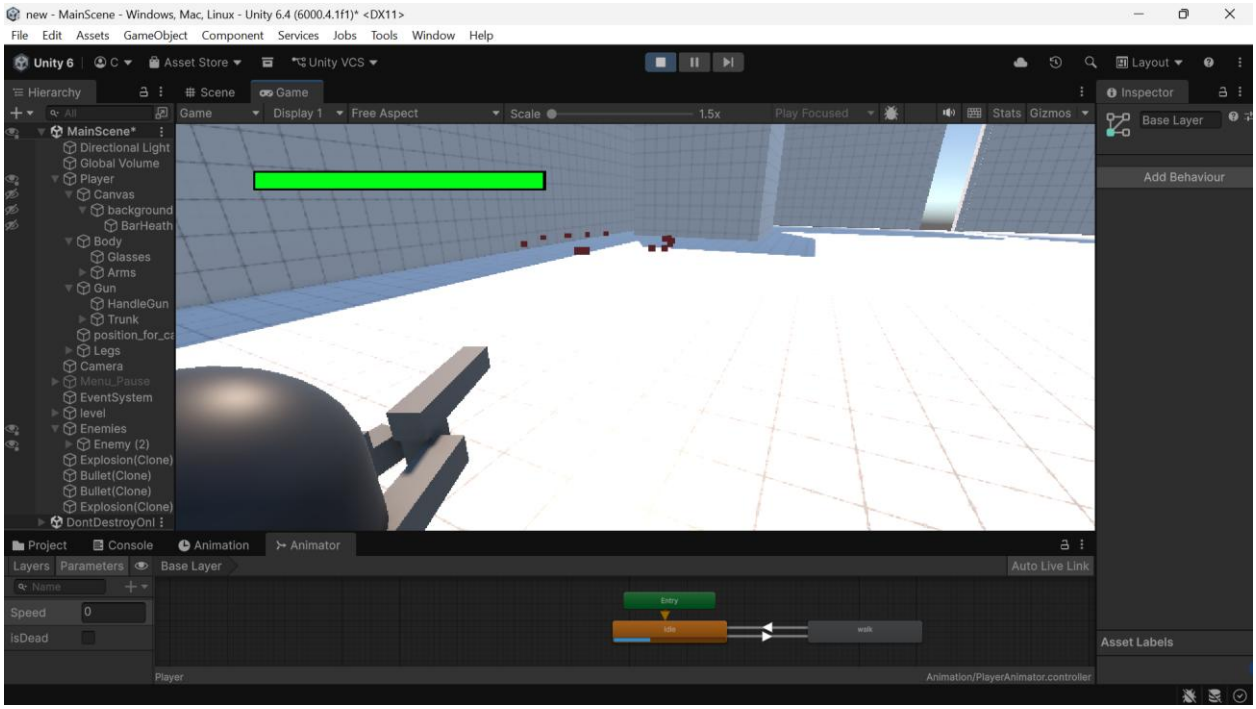


Рисунок 9 — Скриншот Unity анимация исчезновения противника

Практическая работа №5

ModeSwitcher

```
using UnityEngine;

public class ModeSwitcher : MonoBehaviour
{
public GameObject desktopCamera; public GameObject XROrigin;

void Start()
{
// По умолчанию обычный режим
EnableDesktopMode();
}

public void EnableDesktopMode()
{
desktopCamera.SetActive(true); XROrigin.SetActive(false);

Debug.Log("Desktop mode enabled");
}

public void EnableXRMode()
{
desktopCamera.SetActive(false); XROrigin.SetActive(true);

Debug.Log("XR mode enabled");
}
}
```

NetworkPlayer

```
using Unity. etcode; using UnityEngine;
public class        etworkfllayer :    etworkBehaviour
{
void Update()
{
if (IsOwner)
{
```

```
fllayerData data = new fllayerData
{
x = transform.position.x,
y = transform.position.y,
z = transform.position.z
};

SendDataServerRpc(data);
}
}

[ServerRpc]
void SendDataServerRpc(fllayerData data)
{
Debug.Log(
$"DATA: {data.x} {data.y} {data.z}"
);
}
}
```

PlayerController

```
using Unity. etcode; using UnityEngine;


public class fllayerController :  etworkBehaviour
{
[ eader("Movement")] public float speed = 5f;
public float jumpForce = 5f;


[ eader("Components")] public Rigidbody rb;
public MeshRenderer meshRenderer; private bool isGrounded;
// Сетевой цвет
public  etworkVariable<Color> playerColor = new  etworkVariable<Color>();


void Start()
{
playerColor.OnValueChanged += OnColorChanged;


ApplyColor(playerColor.Value);
```

```
}

public override void On etworkSpawn()
{
if (IsOwner)
{
Color randomColor = new Color( Random.value, Random.value,
Random.value
);

SubmitColorServerRpc(randomColor);
}
}

void Update()
{
if (!IsOwner)
return;

Move();
Jump();
}

void Move()
{
float h = Input.GetAxis(" orizontal"); float v = Input.GetAxis("Vertical");

Vector3 move =
new Vector3(h, 0, v);

transform.position +=
move * speed * Time.deltaTime;
}

void Jump()
```

```
{
if (Input.GetKeyDown(KeyCode.Space) fifi isGrounded)
{
rb.AddForce(
Vector3.up * jumpForce, ForceMode.Impulse
);
}
}

private void OnCollisionEnter(Collision collision)
{
if (collision.gameObject.CompareTag("Ground"))
{
isGrounded = true;
}
}

private void OnCollisionExit(Collision collision)
{
if (collision.gameObject.CompareTag("Ground"))
{
isGrounded = false;
}
}

[ServerRpc]
void SubmitColorServerRpc(Color color)
{
playerColor.Value = color;
}

void OnColorChanged(Color oldColor, Color newColor)
{
ApplyColor(newColor);
}
```

```
void ApplyColor(Color color)
{
meshRenderer.material.color = color;
}
}
```

PlayerData

```
using Unity. etcode;

public struct fllayerData : I etworkSerializable
{
public float x; public float y; public float z;

public void          etworkSerialize<T>(BufferSerializer<T> serializer) where T : IReaderWriter
{
serializer.SerializeValue(ref x); serializer.SerializeValue(ref y); serializer.SerializeValue(ref z);
}
}
```

SimpleConnection

```
sing Unity. etcode; using UnityEngine;

public class SimpleConnection : MonoBehaviour
{
public GameObject connectionflanel;

public void Start ost()
{
etworkManager.Singleton.Start ost(); connectionflanel.SetActive(false);
Debug.Log(" OST STARTED");
}

public void StartClient()
{
etworkManager.Singleton.StartClient();
connectionflanel.SetActive(false); Debug.Log("CLIE T STARTED");
}
}
```

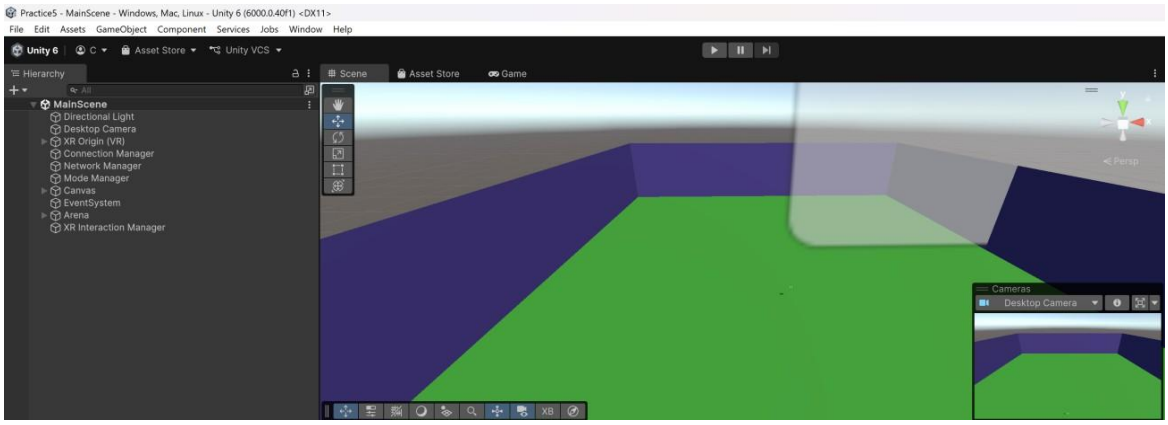


Рисунок 10 — Скриншот Unity

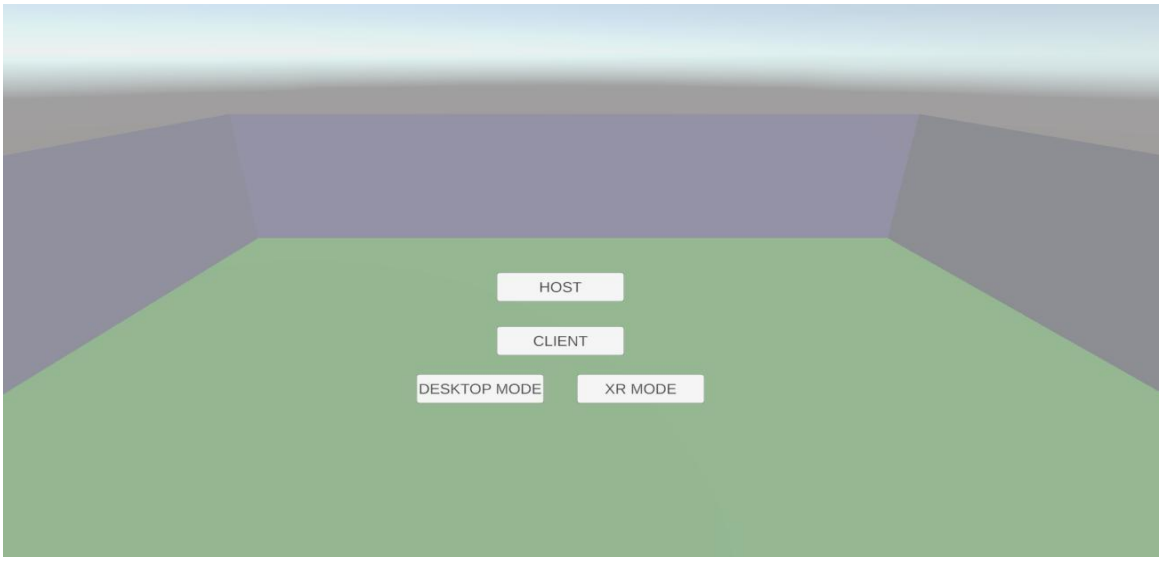


Рисунок 11 — Менюшка выбора режима

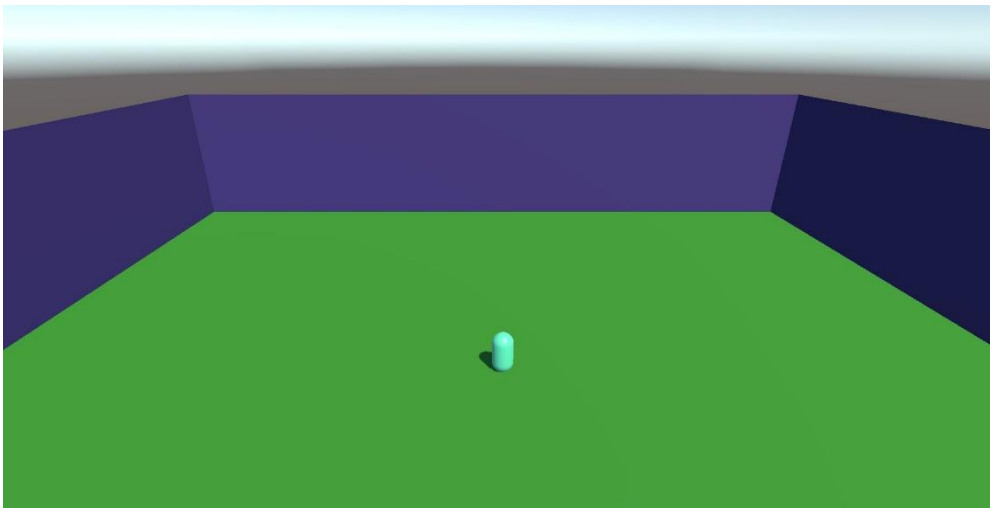


Рисунок 12 — Сервер

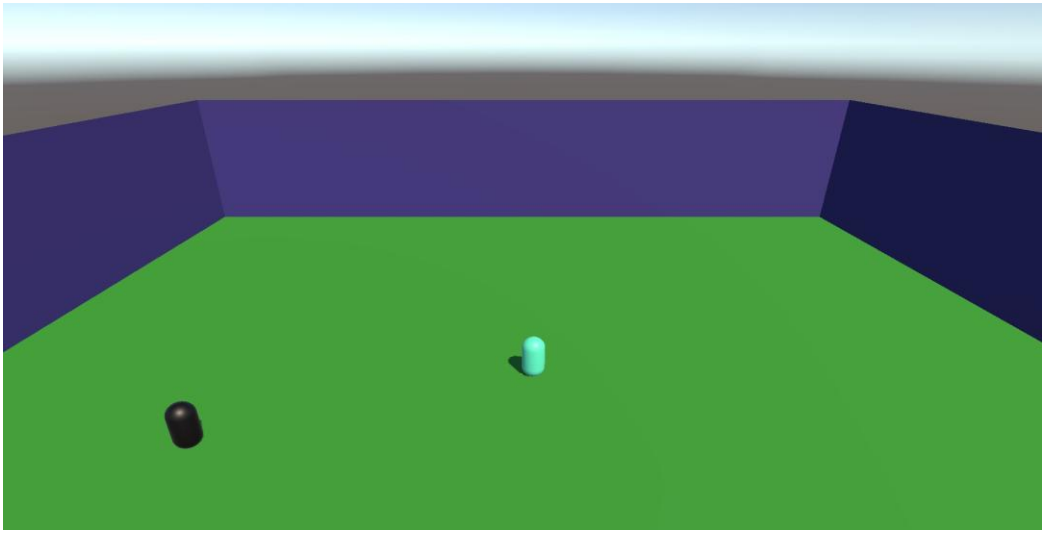


Рисунок 13 — Хост подключенный

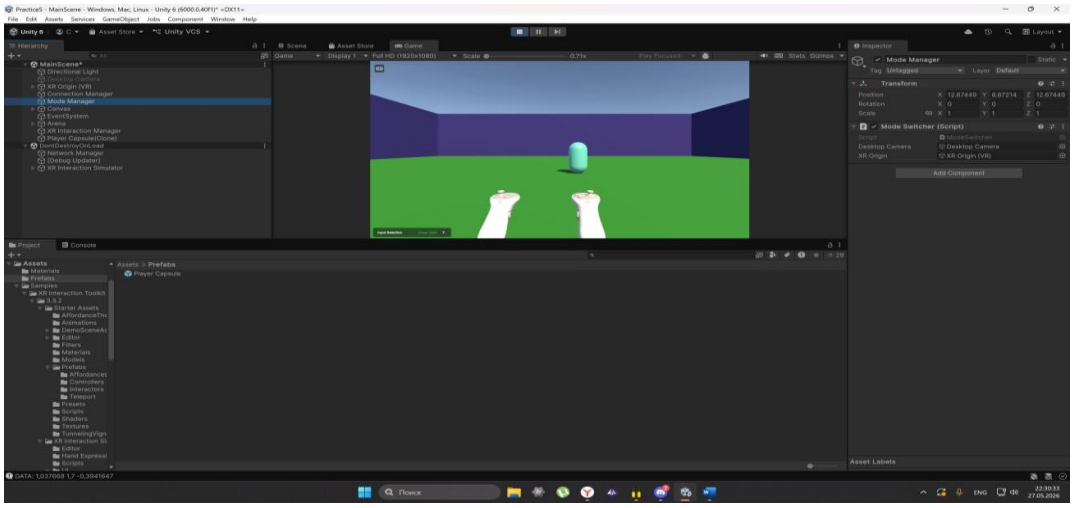


Рисунок 14 — VR режим